Thanks to this problem, it's often good practice to make your error-handling pages as simple as possible to reduce the possibility that errors will occur.

WARNING If your error-handling path generates an error, the user will see a generic browser error. It's often better to use a static error page that always works than a dynamic page that risks throwing more errors. You can see an alternative approach using a custom error handling function in this post: http://mng.bz/0Kmx.

Another consideration when building minimal API applications is that you generally don't want to return HTML . Returning an HTML page to an application that's expecting JSON could easily break it. Instead, the HTTP 500 status code and a JSON body describing the error are more useful to a consuming application. Luckily, ASP.NET Core allows you to do exactly this when you create minimal APIs and web API controllers.

NOTE I discuss how to add this functionality with minimal APIs in chapter 5 and with web APIs in chapter 20.

That brings us to the end of middleware in ASP.NET Core for now. You've seen how to use and compose middleware to form a pipeline, as well as how to handle exceptions in your application. This information will get you a long way when you start building your first ASP.NET Core applications. Later, you'll learn how to build your own custom middleware, as well as how to perform complex operations on the middleware pipeline, such as forking it in response to specific requests. In chapter 5, you'll look in depth at minimal APIs and at how they can be used to build JSON APIs.

Summary